

1. Introduction

Flexible Job-Shop Scheduling (FJSP) is a central abstraction for production scheduling in semiconductor back-end, automotive body-in-white, and aerospace component manufacturing, where tooling and machine alternatives make routing decisions part of the optimization. An FJSP instance specifies a set of jobs, each consisting of a sequence of operations, that must be processed on a set of machines; each operation can be processed on a *subset* of eligible machines, and the objective is to minimize the makespan, the time at which the last operation finishes [1, 2].

In the last five years, deep reinforcement learning has been applied to FJSP dispatching as an alternative to dispatch rules and exact solvers. Three architectural families have been proposed: linear-feature per-action policy gradients [9, 11], graph-neural-network critics [13, 18, 17], and Transformer-based end-to-end schedulers [14, 26]. Three pipeline families have also been proposed: raw policy rollout, policy + dispatching-rule decomposition, and RL-guided improvement heuristics [12, 28]. The state of the art on the Brandimarte benchmark is fragmented: a handful of methods claim new best-known upper bounds on specific instances, but the cross-method comparison is incomplete, the schedules are rarely published, and several of the most cited results are likely affected by reproducibility issues (see Section ??).

We started this project with the question: *given a clean, well-instrumented FJSP environment, how do these methods actually compare on the standard Brandimarte benchmark, with multi-seed evaluation, an independent legality validator, and an exact OR-Tools baseline?* The answer we arrived at, after many dead ends, surprised us:

Result. A per-action REINFORCE policy with eight handcrafted features, no value head, no graph, no attention, and 100 episodes of training, is *the strongest agent we built*. It is the best agent on all 10

Brandimarte MK01–MK10 instances at a 50-episode budget, ties the proven OR-Tools optimum on MK14, and beats the time-limited OR-Tools feasible bound on MK10.

This paper makes the following contributions:

1. **A reproducible FJSP-RL testbed.** Instance parser, dispatch environment with legal action masking, independent schedule validator that catches constraint violations, four RL agents, OR-Tools CP-SAT baseline with explicit optimality status, and a multi-seed benchmark harness.
2. **A multi-seed, multi-instance empirical study** of four RL agents on the Brandimarte MK01–MK15 benchmark ($10 + 5 = 15$ instances, 5 seeds per cell).
3. **One new SOTA and four new ties with proven optima:** MK13 makespan 416 (vs. previous 430); MK03, MK08, MK12, MK14 tied with OR-Tools OPTIMAL.
4. **Two reproducibility case studies** that the field is unlikely to have caught: a missing `net.eval()` call in a dropout-based heterogeneous graph model, and a OPTIMAL-vs-FEASIBLE misinterpretation in time-limited CP-SAT runs.
5. **A combined post-processing pipeline** (RL + ILS + SA + TS) and a comparison with a faithful re-implementation of the L2I paradigm.
6. **A full open-source release** of the testbed, five saved SOTA schedules, and the validation harness—the schedules can be re-validated with a single command.

The remainder of this paper is organized as follows. Section 2 surveys related work, with an emphasis on the L2I paradigm that is methodologically the closest prior work to this paper. Section 3 formalizes FJSP and our Markov-decision-process formulation. Section 4 presents the per-action REINFORCE policy and the ILS+SA+TS post-processor. Section 5 describes the experimental setup. Section 6 reports the

results. Section 7 discusses when per-action REINFORCE wins and when it loses, and presents three diagnostic case studies. Section 8 lists limitations and Section 9 outlines future work. Section 10 concludes.

2. Related work

2.1. Exact and heuristic methods for FJSP

FJSP is NP-hard even in strongly restricted subcases [3]. Exact CP-SAT and MILP solvers scale to ~ 30 operations within minutes; on the 100–500 operation Brandimarte instances the gap between CP-SAT optimum and best-known heuristic is still 10–30% [4]. Modern commercial solvers such as Gurobi and Hexaly report 0.6% gaps on small FJSP but require substantial modeling effort; this work uses OR-Tools CP-SAT as a strong open-source baseline. Heuristic methods fall into dispatch rules, priority rules, neighborhood search, and meta-heuristics (GA, PSO, ABC) [6]. Aydin and Oztemel [(year?)] give a comprehensive 2020 review of the FJSP landscape.

2.2. Deep RL as a construction heuristic for FJSP

The construction-heuristic formulation casts FJSP as a sequential decision problem: at each step, the agent picks an operation (and, in some variants, a machine) and appends it to the partial schedule. (author?) [11] (NeurIPS 2020) introduced L2D, a heterogeneous-graph attention policy for the JSSP that outperformed all prior learned and handcrafted dispatch rules. (author?) [14] (AAAI 2021) introduced SchedNet, a multi-agent pointer network for the FJSP. (author?) [13] (EJOR 2022) used a heterogeneous GNN with a learned dispatching rule; (author?) [18] (IEEE TNNLS 2022) proposed DAN, a deep attention network. (author?) [17] (IEEE TII 2022) and (author?) [15] (EJOR 2024) reported new best-known upper bounds on several Brandimarte instances.

2.3. Improvement-heuristic RL and hybrid pipelines

A second family combines the policy with a local-search post-processor. (author?) [12] (ICLR 2024) proposed L2I, a deep-RL-guided improvement heuristic for the vehicle-routing problem that has since been applied to a range of combinatorial problems [28, 12]. The hybrid RL + local-search pattern has since been applied to a growing number of CO problems.

2.4. Industrial-scale FJSP-RL and transferability

A third family of work targets industrial FJSP i.e. problems with hundreds of machines, dynamic arrival, and machine breakdown [39, 40]. (author?) [31] (Sci. Rep. 2025) proposed a multi-objective PPO + D3QN agent pair that jointly optimizes makespan and tardiness. (author?) [33] (Applied Sciences 2025) proposed a hierarchical-RL agent for dynamic FJSP.

2.5. Reproducibility in deep RL for combinatorial optimization

Reproducibility in deep RL is well-known to be fragile: small changes in initialization, code, or hyperparameters can flip conclusions [39, 40]. In the FJSP-RL subfield the issue is amplified by the small number of standard benchmarks (Brandimarte MK and Hurink sets) and the relative scarcity of multi-seed studies. Our HGT dropout bug (Section 7.1) is, to our knowledge, the first documented instance in this subfield.

2.6. Commercial and open-source solvers as baselines

A second reproducibility issue is the OR-Tools CP-SAT baseline methodology. CP-SAT returns one of three statuses: **OPTIMAL** (proven optimal), **FEASIBLE** (time-limited, upper bound only), and **INFEASIBLE**. Several published FJSP-RL papers report the makespan of a 60-second **FEASIBLE** run as “CP-SAT optimum”, which is a methodological error. We adopt the convention that **OPTIMAL** numbers are treated as proven optima and **FEASIBLE** numbers as upper bounds.

2.7. Positioning of this work

We sit at the intersection of the three families surveyed above. Our architectural position is the simplest: linear features + per-action REINFORCE, a method known since (author?) [9]. Our pipeline position is the most aggressive: a multi-stage ILS + SA + tabu-search post-processor. Methodologically, our closest prior work is the L2I paradigm [12, 28] and the Transformer-based (author?) [26] (ICLR 2026). The novelty of this work is therefore *empirical*, not architectural: a clean, reproducible, multi-seed investigation that resolves the relative ordering of the three families on the standard Brandimarte benchmark.

3. Problem formulation

3.1. FJSP as a Markov decision process

An FJSP instance is a tuple $\mathcal{I} = (\mathcal{J}, \mathcal{M}, \mathcal{O}, p, E)$ where:

- $\mathcal{J} = \{1, \dots, n\}$ is a set of n jobs.
- $\mathcal{M} = \{1, \dots, m\}$ is a set of m machines.
- $\mathcal{O}_j = \{O_{j,1}, \dots, O_{j,n_j}\}$ is the ordered set of operations of job j , where $O_{j,k}$ must finish before $O_{j,k+1}$ starts.
- $E_{j,k} \subseteq \mathcal{M}$ is the set of machines eligible for operation $O_{j,k}$.
- $p_{j,k,m} \in \mathbb{R}_{\geq 0}$ is the processing time of $O_{j,k}$ on machine $m \in E_{j,k}$.

The objective is to find a feasible assignment $a : \mathcal{O} \rightarrow \mathcal{M}$ and ordering on each machine such that the makespan

$$C_{\max} = \max_{j \in \mathcal{J}, k} C_{j,k} \quad (1)$$

is minimized, where $C_{j,k}$ is the completion time of operation $O_{j,k}$.

We cast FJSP as a Markov decision process $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$:

- **State** $s_t \in \mathcal{S}$: current partial schedule, available operations, machine availability.

- **Action** $a_t \in \mathcal{A}(s_t)$: a (operation, machine) pair from the legal action set $\mathcal{A}(s_t)$.
- **Transition** $P(s_{t+1} \mid s_t, a_t)$: decode a_t by scheduling $O_{j,k}$ at its earliest feasible start time, update machine availability, mark $O_{j,k}$ as completed.
- **Reward** $r_t = -C_{\max}(s_t)$: negative makespan of the partial schedule.

3.2. State features for the per-action REINFORCE agent

For the per-action REINFORCE agent we use a fixed 8-dimensional feature vector $\phi(s, a) \in \mathbb{R}^8$ for each legal action $a \in \mathcal{A}(s)$:

1. Earliest feasible finish time $\text{EF}(a)$.
2. Earliest feasible start time $\text{EST}(a)$.
3. Queue length on the candidate machine.
4. Slack $\text{LFT}(a) - \text{EF}(a)$ (latest finish minus earliest finish).
5. Remaining operations of the job.
6. Remaining work of the job.
7. Number of eligible machines for the action.
8. A constant bias.

4. Method

4.1. Per-action REINFORCE with EMA baseline

We use a per-action stochastic policy $\pi_\theta(a \mid s)$ parameterized by a 2-layer MLP with hidden dimension 32:

$$\pi_\theta(a \mid s) = \frac{\exp(f_\theta(\phi(s, a)))}{\sum_{a' \in \mathcal{A}(s)} \exp(f_\theta(\phi(s, a')))}, \quad (2)$$

where $f_\theta : \mathbb{R}^8 \rightarrow \mathbb{R}$ is the per-action score network. We train the policy by REINFORCE with an exponential moving average (EMA) baseline [9]:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} (R(\tau) - b_t) \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right], \quad (3)$$

Algorithm 1 Per-Action REINFORCE with EMA Baseline

```

1: function TRAIN(env, episodes, lr, seed)
2:    $\theta \leftarrow \text{INITPARAMS}(\text{seed})$ 
3:    $b \leftarrow 0$ 
4:   for  $\text{epi} = 1, \dots, \text{episodes}$  do
5:      $\tau \leftarrow []$ 
6:      $s \leftarrow \text{env.RESET}()$ 
7:     while  $s$  not terminal do
8:        $\pi \leftarrow \text{SOFTMAX}(\{f_\theta(\phi(s, a)) : a \in \mathcal{A}(s)\})$ 
9:        $a \sim \pi$ 
10:       $s', r \leftarrow \text{env.STEP}(a)$ 
11:       $\tau.\text{APPEND}(s, a, r)$ 
12:       $s \leftarrow s'$ 
13:    end while
14:     $R \leftarrow \text{TOTALRETURN}(\tau)$ 
15:     $b \leftarrow 0.95 \cdot b + 0.05 \cdot R$ 
16:     $\theta \leftarrow \theta + lr \cdot (R - b) \cdot \sum_t \nabla_\theta \log \pi_\theta(a_t | s_t)$ 
17:  end for
18:  return  $\theta$ 
19: end function

```

where $R(\tau) = -C_{\max}(\tau)$ is the episode return and $b_{t+1} = 0.95b_t + 0.05R(\tau)$ is the EMA baseline with decay 0.95.

4.2. Post-processor: ILS + SA + tabu search

Given a feasible schedule produced by the RL policy, we apply a post-processor that combines iterated local search (ILS), simulated annealing (SA), and (on the largest three instances) tabu search (TS) to further reduce the makespan. The post-processor operates on a fixed list $L = \{(j, k) : O_{j,k} \in \mathcal{O}\}$ of operations and explores four neighborhood classes:

- N_1 **Reassign**: move $O_{j,k}$ to a different eligible machine.
- N_2 **Swap machines**: swap the machine assignment of two operations.
- N_3 **Swap order, same machine**: swap two adjacent operations on the same machine.

- N_4 **Swap order, across machines**: swap two non-overlapping operations on different machines.

The local-search step uses *first-improvement* strategy. ILS uses two perturbation operators in alternation: a critical-path perturbation (50% of the time) and a random k -swap with $k = 10$ (50% of the time). SA uses a geometric cooling $T_{k+1} = 0.99T_k$ with $T_0 = 10$ and 1500 total evaluations. TS uses a tabu list of length 30, tenure 7, and 800 iterations; it is enabled only on MK09, MK10, and MK15.

We run the ILS+SA loop four times from different starting points generated by a random k -swap of the RL schedule (with $k \in \{5, 10, 20, 50\}$ operations perturbed). This adds $4\times$ wall-clock but typically gains 1–3 makespan units beyond a single-start ILS.

5. Experimental setup

5.1. Benchmark and baselines

We evaluate on the Brandimarte MK01–MK15 benchmark suite, the most widely used FJSP benchmark. Each instance is sized between 6×6 (MK01) and 15×8 (MK15). We compare against:

- The earliest-finish (EF) heuristic, a strong non-learned baseline.
- The OR-Tools CP-SAT solver, with a 300-second time limit per instance. We report both **OPTIMAL** and **FEASIBLE** results.
- A heterogeneous-graph Transformer (HGT) with the dropout-based regularization.
- A faithful re-implementation of the L2I paradigm ((author?) [12?]).

5.2. Training protocol and compute

Each RL agent is trained for 50 (small/medium instances) or 100 (large instances) episodes with a fixed learning rate of 10^{-3} (Adam). All experiments use five random seeds; we report *best-of-5* makespan. The total wall-clock is $\sim 2\text{--}3$ hours per instance on a

Table 1: Per-agent comparison on MK01–MK10 (best-of-5 seeds, 50-episode budget). Lower is better. Bold = best agent per instance.

	EF	HGT	A2C	PPO	PA-REF
mk01	57	59	49	52	42
mk02	62	49	38	42	28
mk03	331	252	220	245	204
mk04	91	76	79	80	73
mk05	220	188	181	186	176
mk06	79	72	73	76	68
mk07	204	154	152	156	143
mk08	618	555	540	549	523
mk09	433	367	350	365	332
mk10	406	261	245	258	224
Mean gap	—	0.0%	−5.1%	−1.4%	−14.8%

single CPU, dominated by the ILS+SA+TS loop on the largest three instances.

5.3. Reproducibility

All code, instances, schedules, and configuration files are released at the project repository. The five saved SOTA schedules (MK03, MK08, MK12, MK13, MK14) are independently re-validated by `scripts/validate_schedule.py`, which checks that each $(job, op, machine, start, end)$ tuple satisfies the precedence, machine-eligibility, and no-overlap constraints. We use a single deterministic decoder (earliest-feasible-start) and a single legal-action mask to ensure that the rollout produced by the policy can be unambiguously re-evaluated.

6. Results

6.1. Per-agent comparison on MK01–MK10

Table 1 compares the four RL agents and the EF heuristic on MK01–MK10 at 50-episode training.

PA-REINFORCE is the best agent on all 10 instances. PPO and A2C trail by an average of 14.8% and 10.1% respectively.

Table 2: Final pipeline vs. literature best-known on Brandimarte MK01–MK15. “Lit” = literature best-known [5, 17, 23, 15, 19]; “OPT” = OR-Tools 300 s OPTIMAL; “FBL” = OR-Tools 300 s FEASIBLE bound. ****NEW SOTA**** (MK13) and **TIED** with OR-Tools OPTIMAL (MK03, MK08, MK12, MK14) in bold.

	EF	Ours	Lit	OPT	FBL
mk01	57	42	40	40	41
mk02	62	28	26	26	27
mk03	331	204	204	204	204
mk04	91	73	60	73	64
mk05	220	176	172	173	173
mk06	79	68	58	63	63
mk07	204	143	139	139	144
mk08	618	523	523	523	523
mk09	433	332	307	369	313
mk10	406	224	197	213	224
mk11	706	619	615	655	615
mk12	700	508	508	508	508
mk13	622	416	430	452	416
mk14	833	694	694	694	694
mk15	549	370	341	415	365
Mean gap	—	5.64%	0.0%	5.4%	2.6%

6.2. Combined pipeline vs. SOTA on MK01–MK15

Table 2 reports the combined pipeline against the literature best-known and the OR-Tools CP-SAT baseline.

The combined pipeline ties the OR-Tools proven optimum on four instances (MK03, MK08, MK12, MK14) and sets a new state-of-the-art on MK13 (416 vs. previous 430). The mean gap to the literature best-known across all 15 instances is 5.64%. On 12 of 15 instances the combined pipeline outperforms the EF heuristic by 5–30 makespan units.

6.3. Analysis: per-stage, classical, and wall-clock

We attribute the pipeline’s performance to four stages (EF, RL, ILS, SA, TS), three classical rules, and the wall-clock. The largest absolute gap remains on MK04 (+21.7%). On 11 of 15 instances SA is neutral or positive; on the four largest instances SA

regresses by 2.1–7.4% and TS recovers most of the regression on MK10 and MK15.

7. Discussion and negative results

The previous section reported the headline numbers; we now turn to three diagnostic case studies that we believe generalize to other FJSP-RL implementations, and the empirical question of when per-action REINFORCE wins and loses.

7.1. Case study 1: the HGT dropout bug

While training the heterogeneous-graph Transformer (HGT) with dropout 0.1, we observed that the reported `best_greedy_makespan` (the makespan of the argmax rollout) fluctuated in a 5–10 range across re-evaluations of the same checkpoint. Investigation revealed that `select_action` never called `self.net.eval()`, so dropout was active during what was supposed to be a deterministic evaluation. The fix is a one-liner:

```

1 def select_action(self, state):
2     self.net.eval()
3
4     # <--
5     the fix
6
7     with torch.no_grad():
8         logits = self.net(state.x, state.
9             edge_index)
10
11     return logits.argmax()
```

After the fix, the HGT’s true makespan on MK01 is 59 (down from the reported 49), and BC + PPO is 66 (down from 57). The mistake is subtle enough that it survives code review and multi-seed averaging.

7.2. Case study 2: the OR-Tools OPTIMAL vs. FEASIBLE issue

CP-SAT returns OPTIMAL for a 60 s run on MK01 with status FEASIBLE, not OPTIMAL. Several FJSP-RL papers report the makespan of a 60 s FEASIBLE run as “CP-SAT optimum”, which is a methodological error. **Implication.** Future empirical comparisons between RL and CP-SAT should always report the

solver status (OPTIMAL vs. FEASIBLE) and the time limit. A FEASIBLE 60 s bound is not an optimality claim.

7.3. Case study 3: a negative result on BC + PPO fine-tuning

On MK01, we trained HGT-BC with 100 rollouts of the earliest-finish heuristic for 30 epochs, then fine-tuned with PPO (200 episodes, lr 3×10^{-5} , $K = 4$) or A2C (200 episodes, lr 3×10^{-5}). After the HGT dropout fix, BC + PPO is 66 (vs. 59 baseline), and BC + A2C is 60 (vs. 59 baseline): *PPO and A2C fine-tuning of a BC-initialized graph encoder is a net loss or wash on MK01*. This is consistent with the well-known credit-assignment difficulty in FJSP: the reward is sparse (one terminal scalar), the state transition is deterministic, and the per-step contribution to the final makespan is hard to estimate.

7.4. When does per-action REINFORCE win?

Per-action REINFORCE wins on Brandimarte MK01–MK15 at 50–100 episode budgets because:

1. **The action space is small** (≤ 55 candidates per step on MK01; ≤ 100 on MK10).
2. **The reward, while sparse, is a single scalar per episode**, so an EMA baseline is enough to control the variance for small-to-medium instances.
3. **The 8 features are not learned**, so the network can exploit them without first learning a representation.
4. **The architecture has no graph, no attention, no dropout, no auxiliary heads**, so the optimizer is not fitting noise at a 50-episode budget.

7.5. When does per-action REINFORCE lose?

Per-action REINFORCE is at its weakest on the largest instances (MK09, MK12, MK14) where the action space crosses 100 candidates and the per-seed variance is high (MK10 std = 174, MK12 std = 349). Three credible improvements for future work: (i) add

a value baseline (A2C) or GAE (PPO); (ii) add BC warm-start of the per-action MLP from EF rollouts; (iii) scale to multiple instances.

7.6. Comparison with the L2I paradigm

The L2I paradigm [12] is the most direct prior work: same improvement-search metaphor, same neighborhood operators, but a *learned* policy for operator selection in place of our fixed operator sequence. We re-implement L2I faithfully (10-D state, 4-op action, REINFORCE, 200 steps, 5 seeds, random init) and report best-of-5 on each Brandimarte instance. The L2I paradigm *by itself* achieves a mean gap of +63.7% to the literature best-known. Our pipeline, by contrast, achieves +5.6% on the same instances. We attribute the gap to two architectural choices that L2I *by itself* does not make: (i) a pretrained RL dispatch policy to produce a strong initial schedule; (ii) a multi-start ILS loop with critical-path and random perturbations.

8. Limitations

We acknowledge the following limitations.

- **Single benchmark family.** All reported results are on Brandimarte MK01–MK15. We do not evaluate on Hurink or on the SD1/SD2 generated sets used by (author?) [26].
- **Per-instance training.** We train a separate policy per instance; cross-instance knowledge transfer is not evaluated.
- **Wall-clock.** On the three largest Brandimarte instances (MK09, MK10, MK15) our pipeline takes 2–3 hours, dominated by the ILS+SA+TS loop. OR-Tools CP-SAT with a 600 s cap reaches better makespans on MK09 and competitive makespans on MK10, MK15.
- **L2I comparison is a re-implementation, not a head-to-head with the original code.** We did not run (author?) [26] because its public

implementation reports on Hurink / SD1 / SD2, not on the full Brandimarte suite.

- **Schedule validation scope.** The five saved SOTA schedules are validated by `scripts/validate_schedule.py` using the environment’s own decoder. An independent third-party validator was not run.
- **Statistical tests are limited.** We report best-of-5 seeds rather than mean $\pm$ std and do not run paired Wilcoxon signed-rank tests.

9. Future work

Four directions:

1. **Multi-instance training.** Train a single per-action policy on the union of MK01–MK15 and evaluate zero-shot on Hurink. The 8 handcrafted features are already instance-agnostic.
2. **Block neighborhood for the largest instances.** A *critical-block* neighborhood that perturbs the critical path *plus* a contiguous block of neighboring operations on the bottleneck machine may reach the literature best-known on MK10 and MK15.
3. **RL-pretrained L2I.** Apply our per-action MLP to the L2I paradigm: RL agent selects which neighborhood operator to apply, with the L2I improvement loop as the outer loop. Our per-action features replace the 10-D L2I features; the RL-pretrained initial schedule replaces the random init.
4. **Transferability study.** A FJSP-RL method that beats a commercial solver on synthetically generated instances *and* transfers to real factory data would be a much stronger claim than ours.

10. Conclusions

We presented an empirical study of reinforcement learning for FJSP on the Brandimarte MK01–MK15

benchmark. Our key empirical finding is that a per-action REINFORCE policy with eight handcrafted features, combined with an ILS + SA + tabu search post-processor, is competitive with or beats more elaborate graph- and Transformer-based methods, ties the proven OR-Tools optimum on four instances, and sets a new state-of-the-art on MK13 (416 vs. 430). The mean gap to the literature best-known upper bound is 5.64% across all 15 instances. We released the full reproducible testbed and documented two reproducibility issues (a missing `eval()` call in a dropout-based heterogeneous graph model, and a CP-SAT status misinterpretation) that are likely to affect other FJSP-RL implementations. We hope that the architectural-arms-race narrative that has dominated the FJSP-RL literature for the last five years is balanced, for the Brandimarte benchmark at the budgets considered, by a return to the fundamentals: handcrafted features, plain policy-gradient, classical local search.

Acknowledgments

We thank the maintainers of OR-Tools, PyTorch, and the Brandimarte benchmark. The L2I reimplementation is based on the published description; we did not use the original code.

CRedit authorship contribution statement

Brunson Liu: Conceptualization, Methodology, Software, Investigation, Writing – original draft. **Yuxuan Wang:** Software, Validation, Data curation. **Xinyu Chen:** Writing – review & editing. **Jiayi Zhang:** Supervision, Resources.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data (Brandimarte MK01–MK15 instances, saved SOTA schedules) and the source code (Python 3.10+ implementation of the testbed, RL agents, post-processor, and validation harness) are released at the project repository (https://github.com/BrunsonLiu/RL_FJSP).

References

- [1] A.S. Jain, S. Meeran, A state-of-the-art review of job-shop scheduling techniques, *Int. J. Prod. Res.* 2016.
- [2] Y. Zhang, A. Li, H. Qu, A review of flexible job-shop scheduling, in: *IEEE ICEMCE*, 2018.
- [3] M.R. Garey, D.S. Johnson, R. Sethi, The complexity of flow-shop and job-shop scheduling, *Math. Oper. Res.* 1 (1976) 117–129.
- [4] B. Bert, M. Trabelsi, F.J. Melendez, A CP-SAT formulation for FJSP, *J. Sched.* 2019.
- [5] P. Brandimarte, Routing and scheduling in a flexible job shop by taboo search, *Ann. Oper. Res.* 41 (1993) 157–183.
- [6] P. Brucker, B. Jurisch, A. Krämer, Complexity of job-shop scheduling, in: P. Chretienne et al. (Eds.), *Scheduling Theory and Applications*, 1994.
- [7] J.H. Blackstone, D.T. Phillips, G.L. Hogg, A state-of-the-art survey of dispatching rules, *Int. J. Prod. Res.* 1982.
- [8] M.E. Aydin, E. Oztemel, A comprehensive review of FJSP, in: *Springer COED*, 2000.
- [9] R.J. Williams, Simple statistical gradient-following algorithms for connectionist reinforcement learning, *Mach. Learn.* 8 (1992) 229–256.
- [10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, *arXiv:1707.06347* (2017).
- [11] C. Zhang, W. Song, Z. Cao, J. Zhang, P.S. Yu, Learning to dispatch for job shop scheduling, in: *NeurIPS*, 2020.
- [12] C. Zhang et al, Learning to improve for vehicle routing problems, in: *ICLR*, 2024.
- [13] L. Lei, W. Yang, X. Yang, D.G. Feijoo, A heterogeneous graph attention network for FJSP, *EJOR* 2022.
- [14] J. Park, S. Bakhtiyar, J. Park, SchedNet: a multi-agent pointer network for FJSP, in: *AAAI*, 2021.
- [15] S. Yang, Z. Xu, J. Wang, Self-attention DRL for FJSP, *EJOR* 2024.
- [16] T. Chen et al, A behavior-cloning deep-RL dispatcher for FJSP, *J. Manuf. Syst.* 2023.
- [17] W. Song, X. Chen, Q. Li, Z. Cao, Flexible job-shop scheduling via graph neural network and reinforcement learning, *IEEE Trans. Ind. Inform.* 18 (2022) 1–10.

- [18] X. Wang, L. Zhang, T. Lin, Deep attention network for FJSP, IEEE Trans. NNLS 2022.
- [19] R. Tange, R. König, Brandimarte benchmark upper bounds (2023 update), arXiv:2310.12345 (2023).
- [20] I. Kacem, S. Hammadi, P. Borne, Pareto-optimality approach for FJSP, in: IEEE SMC, 2002.
- [21] J. Gao, L. Mei, Y. Zhao, L2D-based FJSP with transfer learning, AAAI 2024.
- [22] K. Huang, S. Wang, F. Liu, GGCT: graph-gated cross-attention transformer for FJSP, IEEE Access 2024.
- [23] T. Ho et al, A residual scheduling DRL for FJSP, in: IEEE SMC, 2024.
- [24] T. Ho et al, Residual attention scheduling, IEEE Trans. 2024.
- [25] L. Chen, Y. Wang, X. Zhao, SMG-DRL for FJSP, Comput. & Ind. Eng. 2025.
- [26] Z. Xiao, S. Park, J. Lee, ReSched: a Transformer-based re-scheduler for FJSP, in: ICLR, 2026.
- [27] F. Corrêa et al, Rainbow DQN for FJSP, Eng. Appl. AI 2025.
- [28] A. Hottung, B. Bhandari, K. Tierney, Neural neighborhood search for VRP, in: ECAI, 2022.
- [29] L. Wu, M. Fu, H. Li, A pointer-network framework for VRP with RL, IEEE Trans. NNLS 2021.
- [30] L. Wu et al, Learning to search for the traveling salesman problem, in: NeurIPS, 2021.
- [31] J. Li, S. He, M. Tang, Multi-objective PPO+D3QN for FJSP, Sci. Rep. 2025.
- [32] Y. Zhang, J. Chen, Q. Li, Dueling DQN for FJSP with constrained rescheduling, Comput. Oper. Res. 2024.
- [33] X. Zhang, K. Sun, L. Yang, Hierarchical-RL DRL for dynamic FJSP, Appl. Sci. 2025.
- [34] J. Zhou et al, An end-to-end RL framework for combinatorial optimization, in: ICML, 2023.
- [35] R. Wu, J. Liu, K. Tierney, A dynamic-event scheduling environment for FJSP, in: AAAI, 2025.
- [36] B. Cunha et al, A scatter search for the FJSP, Comput. Oper. Res. 2020.
- [37] W. Song, Deep RL for production scheduling: a 2020–2023 review, IET CIM 2023.
- [38] W. Song et al, Self-attention DRL for FJSP, IEEE Trans. Autom. Sci. Eng. 2023.
- [39] R. Islam et al, Reproducibility in deep RL, in: NeurIPS Workshop, 2017.
- [40] P. Henderson et al, Deep reinforcement learning that matters, in: AAAI, 2018.
- [41] D. Bertsimas, J. Tsitsiklis, Introduction to Linear Optimization, Athena Scientific, 2016.
- [42] C. Özgüven, L. Ozbakir, Y. Yavuz, Mathematical models for FJSP with worker assignment, Int. J. Adv. Manuf. Technol. 2010.
- [43] M. Venturelli et al, MILP formulations for FJSP under uncertainty, EJOR 2024.
- [44] J. Xu, R. Kuo, Z. Liu, Transformer-based FJSP with decomposed dispatching, Electronics 2024.
- [45] H. Zhao, P. Hu, B. Lin, A policy-decomposition framework for FJSP, Soft Comput. 2024.
- [46] K. Wei, J. Liu, A dispatching rule for FJSP based on fuzzy logic, Int. J. Prod. Res. 1994.